SECURITY AUDIT REPORT

MPH Solana

DATE

28 November 2025

PREPARED BY

0xTeam.

WEB3 AUDITS

info@0xTeam.Space

0xTeamSpace

0xTeamSpace

Contents

Revision History & Version Control

Version	Date	Author's	Description
1.0	16 Nov 2025	0xkmr, Hema & Gurkirat	Initial Audit Report
2.0	28 Nov 2025	0xkmr, Hema & Gurkirat	Final Audit Report

0xTeam conducted a comprehensive Security Audit on the MPH Solana to ensure the overall code quality, security, and correctness. The review focused on ensuring that the code functions as intended, identifying potential vulnerabilities, and safeguarding the integrity of MPH Solana's operations against possible attacks.

Report Structure

The report is divided into two primary sections:

1. **Executive Summary** : Provides a high-level overview of the audit findings.
2. **Technical Analysis** : Offers a detailed examination of the Solana Program code.

Note :
The manual analysis is exclusively focused on the solana program. The information provided in this report should be used to assess the security, quality, and expected behavior of the code.

1.0 Disclaimer

This is a summary of our audit findings based on our analysis, following industry best practices as of the date of this report. However, it is important to understand that no security audit can guarantee complete protection against all possible security threats. The audit focuses on Solana Program coding practices and any issues found in the code, as detailed in this report. For a complete understanding of our analysis, you should read the full report. We have made every effort to conduct a thorough analysis, but it's important to note that you should not rely solely on this report and cannot make claims against us based on its contents. We strongly advise you to perform your own independent checks before making any decisions. Please read the disclaimer below for more information.

DISCLAIMER: By reading this report, you agree to the terms outlined in this disclaimer. If you do not agree, please stop reading immediately and delete any copies you have. This report is for informational purposes only and does not constitute investment advice. You should not rely on the report or its content, and 0xTeam and its affiliates (including all associated companies, employees, and representatives) are not responsible for any reliance on this report.The report is provided "as is" without any guarantees. 0xTeam excludes all warranties, conditions, or terms, including those implied by law, regarding quality, fitness for a purpose, and use of reasonable care. Except where prohibited by law, 0xTeam is not liable for any type of loss or damage, including direct, indirect, special, or consequential damages, arising from the use or inability to use this report. The findings are solely based on the Solana Program code provided to us.

2.0 Executive Summary

2.1 Overview

0xTeam has meticulously audited the MPH Solana Solana Program project. The primary objective of this audit was to assess the security, functionality, and reliability of the MPH Solana's before their deployment on the blockchain. The audit focused on identifying potential vulnerabilities, evaluating the contract's adherence to best practices, and providing recommendations to mitigate any identified risks. The comprehensive analysis conducted during this period ensures that the MPH Solana is robust and secure, offering a reliable environment for its users.

2.2 Scope

The scope of this audit involved a thorough analysis of the MPH Solana Solana Program, focusing on evaluating its quality, rigorously assessing its security, and carefully verifying the correctness of the code to ensure it functions as intended without any vulnerabilities.

Files in Examination:

Language	Rust
In-Scope	• ~/src • ~/src/Instructions • ~/src/States • ~/src/error.rs • ~/src/lib.rs
Github	https://github.com/zeroexcore/mph/tree/main/packages/sol/programs/ledger/src
Fixed Commit Hash	269627ee02bcba53c95e740a14f74645c2a1f3f6

OUT-OF-SCOPE: External Solana Program libraries, other imported code.

2.3 Audit Summary

Name	Verified	Audited	Vulnerabilities
MPH Solana	Yes	Yes	Refer Section 5.0

2.4 Vulnerability Summary

High	Medium	Low	Informational
0	1	0	1

High Medium Low Informational

2.5 Recommendation Summary

Severity

Issues	🔴 High	🟡 Medium	🟢 Low	🔵 Informational	Total (Σ)
Open					
Resolved		1			**1**
Acknowledged				1	**1**
Partially Resolved					
Total (Σ)		1		1	**2**

- **Open:** Unresolved security vulnerabilities requiring resolution.
- **Resolved:** Previously identified vulnerabilities that have been fixed.
- **Acknowledged:** Identified vulnerabilities noted but not yet resolved.
- **Partially Resolved:** Risks mitigated but not fully resolved.

2.6 Summary of Findings

ID	Title	Severity	Fixed
M-01	Premature Ledger closure can create Loss of Funds to the Users.	Medium	✔
I-01	Arbitrary Allowance Manipulation Enables Unauthorized Token Drainage	Info	📝

✔ - Fixed 🟡 - Partially Fixed ✘ - Not Fixed 📝 - Acknowledged

3.0 Checked Vulnerabilities

We examined the Solana Programs for widely recognized and specific vulnerabilities. Below are some of the common vulnerabilities considered.

Category	Check Items
Source Code Review	➔ Authorization & access-control issues ➔ Incorrect state handling ➔ Inconsistent or unsafe logic paths ➔ Integer arithmetic safety ➔ Panic conditions (unwrap, expect, panic!) ➔ Misuse of unsafe Rust ➔ Duplicate entries or missing validations ➔ Improper error handling ➔ Lack of boundary checks ➔ Logical flaws in flows (stake, transfer, update, withdraw, etc.) ➔ Reentrancy and cross-program Invocation Attacks ➔ Anchor Issues
Functional Testing	➔ Business Logic Validation ➔ Feature Verification ➔ Escrow Security ➔ Token Supply Management ➔ Asset Protection ➔ User Balance Integrity ➔ Data Reliability ➔ Emergency Shutdown Mechanism ➔ Error propagation consistency ➔ Serialization and Deserialization Issues ➔ Account Closure and Rent Exploitation ➔ Token and SPL Token Vulnerabilities ➔ Arithmetic and Precision Attacks

4.0 Techniques , Methods & Tools Used

The following techniques, methods, and tools were used to review all the Solana Program

Structural Analysis:

- Module organization
- Data structure design
- State management logic
- Function responsibilities
- Initialization flows
- Code readability & maintainability

Manual Code Review: Deep manual inspection was performed to evaluate:

- Privileged paths
- Critical logic conditions
- Edge-case behaviors
- Internal consistency
- Potential misuse of Rust types or patterns
- Lifetime, ownership, and borrowing correctness
- Multi-step logic
- Failure handling behavior
- Conditions triggering errors or invalid states
- Arithmetic and boundary tests
- Various user/actor inputs

Note: The following values for “Severity” mean:

- High: Direct and severe impact on the funds or the main functionality of the protocol.
- Medium: Indirect impact on the funds or the protocol's functionality.
- Low: Minimal impact on the funds or the protocol's main functionality.
- Informational: Suggestions related to good coding practices and gas efficiency.

5.0 Technical Analysis

Medium

[M-01] Premature Ledger closure can create Loss of Funds to the Users.

Severity
MEDIUM

Location
Sol/programs/ledger/close_ledger.rs

Issue Description

The close_ledger function makes the Authority close the Ledger account. But the function doesn't perform any necessary checks to make sure all the Users claimed their allowed tokens or not. It creates a Rug pull scenario where authority can prematurely close the ledger account before users claiming all the amount.

Effected Code:

```
assert!(ctx.accounts.ledger.initialised);
    assert_eq!(ctx.accounts.ledger.seed, params.seed);
    assert_eq!(ctx.accounts.ledger.bump, ctx.bumps.ledger);

    if ctx.accounts.ledger.authority != ctx.accounts.authority.key() {
        return Err(LedgerError::Unauthorized.into());
    }

    transfer_checked(
        CpiContext::new_with_signer(
            ctx.accounts.token_program.to_account_info(),
            TransferChecked {
                from: ctx.accounts.ledger_ata.to_account_info(),
                to: ctx.accounts.authority_ata.to_account_info(),
                authority: ctx.accounts.ledger.to_account_info(),
                mint: ctx.accounts.mint.to_account_info(),
            },
```

```
            &[&[
                b"ledger".as_ref(),
                params.seed.as_ref(),
                &[ctx.bumps.ledger],
            ]],
        ),
        ctx.accounts.ledger_ata.amount,
        ctx.accounts.mint.decimals,
    )?;

    Ok(())
}
```

Exploit Scenario:
1. Protocols provide allowance to multiple users.
2. Some Users have not yet claimed all the Tokens.
3. Admin calls the close_ledger and immediately and transfer all remaining tokens to themselves, then the ledger account is closed
4. Users lose the ability to claim.

Recommendation

A minimum time window must be provided to the users to claim all the allowance before closing the ledger account.

Status - Fixed

Info

[I-01] Arbitrary Allowance Manipulation Enables Unauthorized Token Drainage

Severity
INFO

Location
Sol/programs/ledger/upset_entry.rs

Issue Description

The upsert_entry function contains a critical access control vulnerability where the signer passes the allowance(spending limit) and the amount(withdrawal size). So Signer can pass the arbitrary allowance and bypass all the checks and be able to drain all the tokens from the Protocol.

```
if (ctx.accounts.entry.amount + params.amount) > params.allowance {
    return Err(LedgerError::InsufficientAllowance.into());
}

ctx.accounts.entry.amount.add_assign(params.amount.clone());
ctx.accounts.entry.allowance = params.allowance.clone();
```

In the above code snippet both allowance and the amount are user controlled values. Signers can easily pass the Arbitrary values and bypass all these checks and can easily drain the protocol.

Impact / Proof of Concept

1. Loss Of Funds: Signer can repeatedly call upsert_entry with inflated allowance values to extract all tokens from the ledger ATA, bypassing any intended withdrawal limits.
2. The protocol allows the Signer to be updated by the Admin but any single Signer is able to drain all the Tokens.

Effected Code:

```
pub fn handler(ctx: Context<UpsertEntry>, params: UpsertEntryParams) ->
Result<()> {
   assert!(ctx.accounts.ledger.initialised);
   assert_eq!(ctx.accounts.ledger.seed, params.seed);
   assert_eq!(ctx.accounts.ledger.bump, ctx.bumps.ledger);

   if ctx.accounts.ledger.signer != ctx.accounts.signer.key() {
       return Err(LedgerError::InvalidSigner.into());
   }

   if ctx.accounts.ledger.paused {
       return Err(LedgerError::Paused.into());
   }

   if !ctx.accounts.entry.initialised {
       ctx.accounts.entry.seed = params.seed;
       ctx.accounts.entry.initialised = true;
       ctx.accounts.entry.bump = ctx.bumps.entry;
       ctx.accounts.entry.amount = 0;
       ctx.accounts.entry.allowance = 0;
   } else {
     assert_eq!(ctx.accounts.entry.seed, params.seed);
     assert_eq!(ctx.accounts.entry.bump, ctx.bumps.entry);
   }

   if (ctx.accounts.entry.amount + params.amount) > params.allowance {
       return Err(LedgerError::InsufficientAllowance.into());
   }

   ctx.accounts.entry.amount.add_assign(params.amount.clone());
   ctx.accounts.entry.allowance = params.allowance.clone();
 transfer_checked(
       CpiContext::new_with_signer(
           ctx.accounts.token_program.to_account_info(),
           TransferChecked {
               from: ctx.accounts.ledger_ata.to_account_info(),
               to: ctx.accounts.user_ata.to_account_info(),
               authority: ctx.accounts.ledger.to_account_info(),
               mint: ctx.accounts.mint.to_account_info(),
           },
```

```
            &[&[
                b"ledger".as_ref(),
                params.seed.as_ref(),
                &[ctx.bumps.ledger],
            ]],
        ),
        params.amount,
        ctx.accounts.mint.decimals,
    )?;

    Ok(())
}
```

Recommendation

Remove User-Controlled Allowance Updates, Possible fixes could include:

- The params.allowance should NOT be accepted from users in the UpsertEntryParams struct.
- Allowance should only be modifiable by the ledger authority through a separate privileged instruction

Status - Acknowledged

6.0 Auditing Approach and Methodologies Applied

The Rust-based code was audited using a comprehensive, security-focused methodology designed to ensure correctness, reliability, and robust protection against logical and implementation-level vulnerabilities. The review covered structural, functional, and security aspects of the Rust codebase to ensure adherence to industry-standard best practices.

Code quality and structure:

A detailed examination of the codebase was performed to evaluate:

- Overall architecture and module organization
- Readability, maintainability, and consistency
- Correctness of state-handling logic
- Safe use of Rust's ownership, borrowing, and lifetimes
- Proper separation of concerns within functions and modules

This ensures that the code follows clean design patterns and avoids unnecessary complexity.

Security vulnerabilities:

We conducted an in-depth manual security review to detect vulnerabilities specific to Rust-based logic, including:

- Missing or weak authorization checks
- Incorrect state transitions
- Logic inconsistencies or unintended execution paths
- Unsafe arithmetic operations
- Unhandled edge cases
- Use of unwrap, expect, or panic-triggering patterns
- Potential misuse of unsafe Rust
- Inadequate input validation

The analysis emphasized robust defensive coding practices that ensure safe and predictable behavior under all conditions.

Documentation and comments:

Documentation, code comments, and inline explanations were examined to verify:

- Accuracy of described logic
- Alignment between intent and implementation
- Clarity for future maintainers

Well-documented code reduces the risk of misinterpretation and accidental introduction of vulnerabilities.

Compliance with best practices:

The code was assessed against established Rust engineering and secure-coding principles, including:

- Zero-panic, zero-unsafe Rust guidelines
- Predictable error handling with Result types
- Clear separation between computation and state mutation
- Defensive checks on all external user inputs
- Consistent error propagation and pattern matching

Adhering to these principles enhances reliability, auditability, and long-term maintainability.

6.1 Code Review / Manual Analysis

A thorough line-by-line manual review of the Rust codebase was conducted to identify vulnerabilities not detectable by automated tools. This process included:

- Evaluating logic correctness
- Confirming that all critical checks and validations are present
- Ensuring safe Rust patterns are consistently used
- Verifying the correctness of state transitions and boundary conditions

Manual analysis allowed us to uncover deeper logic flaws and edge-case vulnerabilities that automated scanners may overlook.

7.0 Limitations on Disclosure and Use of this Report

This report contains information concerning potential details of the MPH Solana Project and methods for exploiting them. 0xTeam recommends that special precautions be taken to protect the confidentiality of both this document and the information contained herein. Security Assessment is an uncertain process, based on past experiences, currently available information, and known threats. All information security systems, which by their nature are dependent on human beings, are vulnerable to some degree. Therefore, while 0xTeam considers the major security vulnerabilities of the analysed systems to have been identified, there can be no assurance that any exercise of this nature will identify all possible vulnerabilities or propose exhaustive and operationally viable recommendations to mitigate those exposures. In addition, the analysis set forth herein is based on the technologies and known threats as of the date of this report. As technologies and risks change over time, the vulnerabilities associated with the operation of the MPH Solana **Rust-based Solana Program** described in this report, as well as the actions necessary to reduce the exposure to such vulnerabilities, will also change. 0xTeam makes no undertaking to supplement or update this report based on changed circumstances or facts of which 0xTeam becomes aware after the date hereof, absent a specific written agreement to perform the supplemental or updated analysis. This report may recommend that 0xTeam use certain software or hardware products manufactured or maintained by other vendors. 0xTeam bases these recommendations upon its prior experience with the capabilities of those products. Nonetheless, 0xTeam does not and cannot warrant that a particular product will work as advertised by the vendor, nor that it will operate in the manner intended. This report was prepared by 0xTeam for the exclusive benefit of MPH Solana and is proprietary information. The Non-Disclosure Agreement (NDA) in effect between 0xTeam and MPH Solana governs the disclosure of this report to all other parties including product vendors and suppliers.